JWT 게시판 구현 보고서

맛집 후기 게시판 · React + Express + SQLite

01 서비스 소개와 구현 범위

학번 202312906 **이름** 박준이 **제출일** 2026. 10. 28.

서비스 이름 맛집 후기 게시판

GitHub 저장소 https://github.com/junyi11111/matjip-review-board

소스 기준 브랜치 main / 커밋 691db35 ※ 최종 제출 SHA 는 LMS 에 제출

구분	항목	버전 / 값
실행 환경	OS / 브라우저	macOS 26.5.2 / Chrome 154
런타임	Node.js / npm	v26.8.2 / 11.19.1
프론트	React / Vite	19.3.0 / 8.3.4
백엔드	Express / jsonwebtoken	5.3.0 / 9.0.3
저장소	SQLite	backend-express/data/board.sqlite
주소	백엔드 / 프론트(빌드본)	127.0.0.1:8000 / 127.0.0.1:4173

서비스 소개와 선택 이유

맛집 후기를 공유하는 게시판이다. 누구나 로그인 없이 글을 읽을 수 있고, 로그인한 사용자만 글을 작성하며, 자신이 쓴 글만 수정·삭제할 수 있다. 테스트 계정은 Alice 와 Bob 을 사용한다.

React 와 Express 는 모두 JavaScript 를 쓰므로 두 계층에서 언어를 따로 익히지 않아도 되고 실행 절차가 단순하다고 판단해 선택했다. 제공된 시작 코드에서 게시판 CRUD 와 SQLite 저장, 기본 화면은 유지하고 JWT 발급·검증, 작성자 권한 검사, 프론트 로그인 연결과 응답 코드별 오류 처리를 구현했다. 그 밖에 시작 코드의 안내 문구를 정리하고 500 오류가 서버 로그에 남도록 보완했다.

기능별 구현 현황

기능	완료	관련 파일	비고
공개 목록과 상세	완료	app.js(제공 유지), main.jsx	토큰 없이 200 (T01, T16)
로그인과 JWT 발급	완료	auth.js (loadConfig, issueToken)	HS256, sub·iss·aud·exp, TTL 90 초
인증과 내 정보 조회	완료	auth.js (currentUserId)	/me 는 id·name·email 만, 401 세 종류 구분

기능	완료	관련 파일	비고
본인 글 작성·수정·삭제	완료	auth.js (assertOwner), app.js 라우트	타인 글 403, authorId 주입 400
만료·로그아웃·오류 표시	완료	api.js, main.jsx	401 은 로그아웃, 403 은 로그인 유지
DB 데이터 유지	완료	data/board.sqlite	재시작 전후 동일 (T15)

구조와 변경 내역

전체 구조

```
브라우저 (React, 127.0.0.1:4173)
  │ fetch — api.js, 토큰은 모듈 변수 session(메모리)
  ▼ Vite preview 프록시
API 서버 (Express, 127.0.0.1:8000)
  │ 라우트(app.js) → 인증·권한(auth.js) → 저장소(store.js)
  ▼
SQLite 파일 (backend-express/data/board.sqlite)
```

개발 중에는 npm run dev(5173)를 썼고, 최종 확인은 npm run build 후 npm run preview(4173)로 했다.

요청 흐름

로그인. 폼의 email·password → 프론트가 clearSession() 후 POST /api/auth/login(헤더 없음) → 서버가 제공된 검증 함수로 계정을 확인하고 JWT 를 발급해 200 으로 accessToken·expiresIn·user 반환(실패 시 401 INVALID_CREDENTIALS) → 프론트가 토큰과 user 를 메모리에만 저장.

게시글 수정. Authorization: Bearer <JWT>를 붙여 PUT /api/posts/{id} → ① 인증(401) → ② ID 형식(400) → ③ 글 존재(404) → ④ 작성자 권한(403) → ⑤ 입력 검증(400) → ⑥ DB 변경 후 200. 공개 조회는 토큰 없이 동작한다.

토큰 전송 여부

요청	Authorization 헤더
POST /api/auth/login	없음 (요청 전 clearSession())
/me, 글 등록·수정·삭제	Bearer <JWT>
공개 조회 (로그아웃)	없음 (토큰 없으면 키 자체가 생기지 않음)
공개 조회 (로그인)	Bearer <JWT> — 서버가 읽지 않아 동작 차이 없음
새로고침 이후	없음 (메모리 토큰 소실)

변경한 파일과 함수

파일 / 함수	원래 역할	변경 내용과 이유	출처
auth.js / loadConfig (신규)	없음	시작 시 JWT_SECRET(빈 값·예시 값·64 자리 16 진수)과 TOKEN_TTL 검사 후 32 바이트 키·iss·aud·ttl 생성. 잘못된 키로 서버가 뜨는 것을 막기 위해	AI 작성, 본인 검토 후 반영

파일 / 함수	원래 역할	변경 내용과 이유	출처
auth.js / issueToken	501 반환	빈 페이로드에 sub·iss·aud·exp 를 옵션으로만 지정해 HS256 발급. 불필요한 정보가 토큰에 들어가지 않게	AI 작성, 본인 검토 후 반영
auth.js / currentUserId	항상 ID 1 반환	Bearer 확인 → jwt.verify → exp·sub 확인 → findPublic 으로 계정 확인 후 조회된 ID 반환. 서버가 확인한 사용자만 넘기기 위해	AI 작성, 본인 검토 후 반영
auth.js / assertOwner	본문 비어 있음	검증된 ID 와 글의 authorId 가 다르면 403. 버튼을 숨겨도 API 직접 호출이 가능하므로	AI 작성, 본인 검토 후 반영
app.js / 오류 미들웨어	500 을 출력 없이 응답	500 일 때만 console.error 로 메서드·경로·오류 출력(헤더·본문 제외). 원인 추적을 위해	AI 작성, 방향성 본인 선택
app.js / app.listen	"인증 TODO 미완성" 로그	서비스 시작 로그로 교체	AI 참고, 문구는 본인
.env.example / .env	TODO 와 자리표시 키	키 생성 안내로 교체, 실제 키는 난수로 생성해 .env 에만 보관(git 제외)	AI 작성, 본인 검토 후 반영
api.js / authorizationHeaders	항상 {}	토큰이 있을 때만 Bearer 객체 반환. Bearer undefined 방지	AI 작성, 본인 검토 후 반영
api.js / request	401 에도 세션 유지	401 이면 clearSession(), code 별 안내 문구, JSON 아닌 응답도 안전 처리	AI 작성, 본인 검토 후 반영
api.js / login	바로 예외	clearSession() 후 로그인 요청, 토큰을 메모리에만 저장	AI 작성, 본인 검토 후 반영
api.js / canEdit	항상 true	로그인 ID 와 authorId 가 같을 때만 true	AI 작성, 본인 검토 후 반영
main.jsx / App	데모 배너, 테스트 계정 미리 입력	404 처리, 로그인 확인, 로그아웃 시 폼 초기화, 문구 교체, 비밀번호 안내 삭제	AI 초안 후 본인 수정
index.html / <title>	시작 코드 제목	서비스 이름으로 교체	AI 작성, 이름은 본인
.gitignore	9 줄	*.sqlite, .env.*(!.env.example), 로그·에디터 설정 추가	AI 작성항목은 본인

근거: docs/worklog.md

로컬 실행 기록

단계	명령	결과
새 폴더 설치	git clone → 각 폴더 npm ci	성공. 빈 폴더에 clone 한 뒤 README 명령만으로 재현됨. npm ci → .env 생성과 키 설정 → npm start(health 200, data/board.sqlite 자동 생성) → npm run build → npm run preview(4173 접속, 목록 200, Alice 로그인·등록 201). 막힌 단계 없음.
환경설정	.env.example → .env, JWT_SECRET 난수 설정	git check-ignore 로 .env 제외 확인
백엔드	npm start	GET /api/health → 200 {"status":"ok"}
프론트 빌드·실행	npm run build → npm run preview	4173 접속, 빌드 결과(/assets/index-*.js) 수신 확인
API 연결	curl 127.0.0.1:4173/api/health	200, X-Powered-By: Express (프록시 전달 확인)
재시작 후 유지	백엔드 종료 → 재실행	재시작 전후 GET /api/posts 응답 동일 (T15)

핵심 코드 설명

본 장의 동작은 모두 4 장 T01~T17 에서 실제 요청으로 확인했다. 줄 번호는 커밋 691db35 기준이다.

3-1 로그인과 JWT 발급

파일 / 함수: backend-express/src/auth.js 의 loadConfig(), issueToken(user)

로그인 라우트 app.post("/api/auth/login")은 제공 코드 그대로 사용했다.

auth.js (5~33 행)

```
const EXAMPLE_SECRET = "REPLACE_WITH_RANDOM_64_HEX_CHARACTERS";
function loadConfig() {
  const secret = process.env.JWT_SECRET ?? "";
  if (!secret) throw new Error("JWT_SECRET 이 비어 있습니다.");
  if (secret === EXAMPLE_SECRET) throw new Error("JWT_SECRET 이 제공 예시 값입니다.");
  if (!/^[0-9a-fA-F]{64}$/.test(secret))
    throw new Error("JWT_SECRET 은 64 자리 16 진수여야 합니다.");
  const ttl = process.env.TOKEN_TTL ?? "90";
  if (!/^[1-9]\d*$/.test(ttl)) throw new Error("TOKEN_TTL 은 양의 정수여야 합니다.");
  return { key: Buffer.from(secret, "hex"),
    issuer: process.env.JWT_ISSUER || "webservice-class",
    audience: process.env.JWT_AUDIENCE || "board-api", ttl: Number(ttl) };
}
const config = loadConfig();
```

```
export function issueToken(user) {
  return jwt.sign({}, config.key, {
    algorithm: "HS256", subject: String(user.id),
    issuer: config.issuer, audience: config.audience, expiresIn: config.ttl });
}
```

설정 검사. loadConfig()는 서버 시작 시 한 번 실행된다. 키가 빈 값·예시 값이거나 64 자리 16 진수가 아니면 오류를 던져 서버가 아예 시작되지 않는다. 잘못된 키로 동작하는 상태를 막기 위해서다. 오류 메시지에 키 값을 넣지 않아 로그로 키가 새지 않게 했다. 예시 값 검사를 형식 검사보다 먼저 둔 것은 "예시 값이니 새로 만들라"는 더 정확한 안내를 내기 위해서다. TOKEN_TTL 도 양의 정수인지 확인하는데, app.js 가 응답의 expiresIn 을 따로 계산하기 때문에 잘못된 값이면 응답과 실제 exp 가 어긋날 수 있어서다.

입력·처리·출력. 입력은 JSON 의 email·password 다. app.js 가 fields()로 입력을, verifyCredentials()로 계정을 확인한 뒤 issueToken()을 호출한다. 비밀번호 검증은 제공된 users.js 함수를 그대로 썼다.

토큰 구성. jwt.sign 의 첫 인자를 빈 객체로 두고 클레임을 옵션으로만 전달했다. 따라서 토큰에는 sub(문자열 ID), iss, aud, exp 와 라이브러리가 넣는 iat 만 들어가고 비밀번호·해시·이메일은 들어가지 않는다. 서명은 jsonwebtoken 에 맡겼다. iss·aud 기본값은 docs/02_API 명세.md 의 값을 따랐다.

실패 조건. 입력 형식 오류는 400 BAD_INPUT, 틀린 비밀번호와 없는 계정은 모두 401 INVALID_CREDENTIALS 다. 둘을 같은 응답으로 돌려주는 것은 어떤 이메일이 가입되어 있는지 외부에서 알아내지 못하게 하기 위해서다. "abc"처럼 @가 없는 문자열은 명세상 길이 규정만 있어 형식 검사를 통과하고 계정 조회에서 401 이 된다. 이 동작을 유지하려고 app.js 에 이메일 형식 검사를 추가하지 않았다.

3-2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-react/src/api.js 의 authorizationHeaders(), request(), login(), canEdit()

```
api.js (15~20 행)
function authorizationHeaders() {
  return session.accessToken
    ? { Authorization: `Bearer ${session.accessToken}` } : {};
}
```

```
request() 내부 (31~44 행)
const data = response.status === 204 ? null : await response.json().catch(() => null);
if (!response.ok) {
  if (response.status === 401) clearSession();   // 403 은 로그인 유지
  const error = new Error(MESSAGES[data?.code] || data?.message ||
    `요청을 처리하지 못했습니다. (HTTP ${response.status})`);
  error.status = response.status; error.code = data?.code;
  throw error;
}
```

```
login(), canEdit() (47~61 행)
export async function login(email, password) {
  clearSession();                                  // 기존 토큰이 붙지 않게
  const data = await request("/auth/login", { method: "POST",
    body: JSON.stringify({ email, password }) });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
export function canEdit(post) {
  return session.user !== null && post.authorId === session.user.id;
}
```

토큰 보관. 토큰은 모듈 변수, 즉 브라우저 메모리에만 둔다. localStorage·sessionStorage 는 쓰지 않았다. 새로고침하면 로그인이 풀리는데 이는 과제 기준상 정상이다. 게시글은 서버 SQLite 에 있으므로 새로고침·재시작과 무관하게 유지된다. 로그인 상태 유지와 데이터 유지는 다른 문제다.

요청 전 clearSession(). 다른 계정으로 로그인할 때 기존 토큰이 로그인 요청에 붙지 않게 하고, 실패 시 이전 사용자로 로그인된 화면이 남지 않게 한다. 실패한 요청을 성공처럼 보이지 않게 하는 처리다.

헤더 생성. 토큰이 있을 때만 Authorization 키를 만들고 없으면 빈 객체를 반환한다. 빈 객체를 펼치면 키가 생기지 않으므로 Bearer undefined 가 나갈 수 없다. 로그인 상태에서는 공개 조회에도 헤더가 함께 나가지만 서버가 읽지 않아 동작 차이가 없고, 로그아웃 상태에서는 붙지 않으므로 공개 조회 요건을 충족한다.

응답 코드별 화면 처리.

HTTP / code	화면 처리	로그인 상태
200 / 201	목록·상세·사용자 상태 갱신	유지
204	JSON 을 파싱하지 않고 목록·선택 갱신	유지
400 BAD_INPUT	서버 문구를 그대로 표시, 입력값 유지	유지
401 INVALID_CREDENTIALS	"이메일 또는 비밀번호가 올바르지 않습니다."	해제

HTTP / code	화면 처리	로그인 상태
401 TOKEN_REQUIRED	"로그인이 필요합니다..."	해제
401 INVALID_TOKEN	"로그인 정보가 올바르지 않습니다..."	해제
401 TOKEN_EXPIRED	"로그인 시간이 만료되었습니다..."	해제
403 FORBIDDEN	"본인이 작성한 후기만 수정·삭제할 수 있습니다."	**유지**
404 NOT_FOUND	안내 후 상세·수정 해제, 목록 갱신	유지

400 만 서버 문구를 그대로 쓴다. 제목·내용·로그인 입력·ID 등 상황이 여러 가지여서 어느 칸이 문제인지는 서버가 더 정확히 알기 때문이다. 403 에서 로그아웃시키지 않는 것은 토큰이 여전히 유효하기 때문이다. 401 은 안내만 하고 입력값을 남겨 재로그인 후 바로 제출할 수 있게 했다.

작성자별 버튼과 로그아웃. canEdit 은 로그인 ID 와 글의 authorId 를 비교한다. 비로그인 사용자에게는 로그인 안내를 띄우고 등록 버튼을 비활성화하며, save() 첫 줄에서도 로그인 상태를 다시 확인해 버튼 우회에 대비했다. 다만 화면 처리는 편의일 뿐이고 실제 차단은 서버의 assertOwner 가 한다. 로그아웃은 세션·화면·수정 모드·입력 폼을 함께 비운다.

요구사항 외 개선. 실패 응답 본문이 JSON 이 아닐 때(백엔드가 꺼져 Vite 프록시가 text/plain 을 반환하는 경우 등) response.json()이 예외를 던지던 문제를 .catch(() => null)로 보완하고 HTTP 상태를 포함한 문구를 표시하도록 고쳤다.

서버 인증과 작성자 권한

3-3 서버 JWT 검증

파일 / 함수: backend-express/src/auth.js 의 currentUserId(req)

```
auth.js (34~65 행)
export function currentUserId(req) {
  const match = /^Bearer\s+(\S+)$/i.exec(req.get("Authorization") ?? "");
  if (!match) throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다.");
  let payload;
  try {
    payload = jwt.verify(match[1], config.key, {
      algorithms: ["HS256"], issuer: config.issuer, audience: config.audience });
  } catch (error) {
    if (error instanceof jwt.TokenExpiredError)
      throw new ApiError(401, "TOKEN_EXPIRED", "로그인이 만료되었습니다.");
    if (error instanceof jwt.JsonWebTokenError)
      throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
    throw error;                          // JWT 오류가 아니면 감추지 않음
  }
  // jsonwebtoken 은 exp 가 없으면 만료 검사를 건너뛰므로 직접 확인
  if (typeof payload !== "object" || typeof payload.exp !== "number" ||
      typeof payload.sub !== "string" || !/^[1-9]\d*$/.test(payload.sub))
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  const user = findPublic(Number(payload.sub));
  if (!user) throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  return user.id;
}
```

입력과 출력. 입력은 Authorization 헤더 하나, 출력은 검증을 통과한 사용자 ID(숫자)다. 보호 API 의 첫 단계에서 호출되므로 여기서 돌려준 ID 가 /me 응답과 글 등록·수정·삭제에 그대로 쓰인다. 요청 본문이나 화면 상태가 아니라 서버가 검증한 결과만 이후 처리로 넘어간다.

단계	검사	실패 시
1	Bearer <토큰> 형식 (i 플래그: 인증 방식 이름은 대소문자 무관)	401 TOKEN_REQUIRED
2	jwt.verify — 알고리즘 HS256 고정, 서명·만료·iss·aud	401 TOKEN_EXPIRED / INVALID_TOKEN
3	exp 존재, sub 가 양의 정수 문자열	401 INVALID_TOKEN
4	findPublic 으로 실제 계정 확인 후 조회된 user.id 반환	401 INVALID_TOKEN

2 단계. algorithms: ["HS256"]을 고정해 alg: none 이나 다른 알고리즘 서명이 거부된다. 내용만 읽는 decode 는 서명을 확인하지 않아 변조를 거르지 못하므로 쓰지 않았다. 예외 순서가 중요한데, TokenExpiredError 는 JsonWebTokenError 를 상속하므로 만료를 먼저 분기하지 않으면 만료 토큰까지 INVALID_TOKEN 으로 묶인다. JWT 오류가 아닌 예외는 다시 던진다. 예상하지 못한 오류를 401 로 감추면 원인을 찾기 어려워지기 때문이다.

3 단계. jsonwebtoken 은 exp 가 없는 토큰이면 만료 검사를 건너뛰고 통과시킨다. 만료되지 않는 토큰이 생기므로 직접 확인한다.

4 단계. 서명이 유효해도 그 사이 사라진 계정일 수 있어 조회로 확인한다. 반환값은 토큰의 sub 가 아니라 조회된 user.id 다. 토큰 값을 그대로 신뢰하지 않기 위해서다.

모든 401 에는 WWW-Authenticate: Bearer 가 붙는다. app.js 의 오류 미들웨어가 제공 코드 단계부터 하던 동작이라 확인만 했다. /me 도 이미 findPublic 으로 id·name·email 만 반환하고 보호 라우트가 첫 줄에서 currentUserId 를 호출하므로 app.js 는 수정하지 않았다.

3-4 작성자 권한 검사

파일 / 함수: backend-express/src/auth.js 의 assertOwner(userId, authorId)

호출 위치는 app.js 의 PUT·DELETE 라우트이며 제공 코드 그대로다.

```
auth.js (66~70 행)
export function assertOwner(userId, authorId) {
  if (userId !== authorId)
    throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");
}
```

```
app.js (69~83 행, 수정하지 않음)
app.post("/api/posts", (req, res) => {
  const userId = currentUserId(req);
  const p = store.create(fields(req.body, { title: 100, content: 5000 }), userId);
  res.status(201).location(`/api/posts/${p.id}`).json(p);
});
app.put("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req), id = postId(req), p = getPost(id);
  assertOwner(userId, p.authorId);
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
});
```

무엇을 비교하는가. 두 값 모두 서버가 가진 값이다. userId 는 토큰 검증과 계정 조회를 마친 숫자 ID, authorId 는 SQLite 의 author_id 컬럼 값이다. 요청 본문이나 화면에서 온 값은 들어가지 않는다. 다르면 예외를 던지므로 뒤의 store.update·store.remove 가 실행되지 않아 DB 는 변경되지 않는다. 엄격 비교(!==)를 쓴 것은 양쪽이 모두 숫자이기 때문이며, 실제로 타입을 확인했다. 한쪽이 문자열이었다면 본인인데도 403 이 났을 것이고 이는 "본인 글 수정 200" 시험에서 바로 드러난다.

작성자 위조 차단. 등록 시 authorId 는 요청 본문이 아니라 currentUserId(req)의 결과로 정해진다. 본문은 fields(req.body, {title, content})를 거치는데 허용하지 않은 키가 있으면 400 BAD_INPUT 이 난다. 따라서 authorId·userId·id 를 넣어 남의 이름으로 쓰려는 시도는 글이 만들어지기 전에 거부된다(T12).

기존 값 유지. store.update 의 SQL 은 UPDATE posts SET title=?, content=?, updated_at=?이라 author_id 와 created_at 은 그대로 남는다.

401 과 403 의 차이.

구분	401	403
의미	누구인지 확인 불가 (인증 실패)	확인됨, 권한 없음
code	TOKEN_REQUIRED / INVALID_TOKEN / TOKEN_EXPIRED	FORBIDDEN
서버 위치	currentUserId	assertOwner

구분	401	403
프론트 처리	clearSession() → 로그아웃, 재로그인 안내	로그인 유지, 권한 안내만

판단 순서. PUT 라우트는 인증(401) → ID 형식(400) → 글 존재(404) → 권한(403) → 입력 검증(400) → DB 변경 순이다. 이 순서가 겹친 요청의 결과를 정한다.

겹친 요청	결과	이유
토큰 없이 없는 글 수정	401 TOKEN_REQUIRED	인증이 글 존재보다 먼저
정상 토큰으로 없는 글 수정	404 NOT_FOUND	인증 통과 후 글 없음
Bob 이 Alice 글에 공백 제목	403 FORBIDDEN	권한이 입력 검증보다 먼저
Alice 가 자기 글에 공백 제목	400 BAD_INPUT	권한 통과 후 입력 검증

예외로 ID 형식 검사는 인증과 글 존재 확인 사이에서 400 을, 깨진 JSON 은 express.json()이 라우트 전에 400 을 낸다. 둘 다 명세가 허용하는 동작이다.

프론트 버튼과의 관계. 버튼을 숨겨도 curl 로 같은 요청을 보낼 수 있으므로 서버의 assertOwner 가 실제 방어선이다. 이를 확인하려고 화면이 아니라 API 를 직접 호출해 Bob 이 Alice 글을 수정·삭제하도록 시험했다(T07, T08).

테스트 결과

항목	내용
실행 환경	macOS 26.5.2 / Node v26.8.2 / Chrome 154 / 백엔드 8000, 프론트 preview 4173
로컬 배포	npm ci → npm run build → npm run preview 로 빌드본 실행 후 확인
일시 / 코드 버전	2026-10-28 / 커밋 691db35
토큰 TTL	일반 90 초 / 만료 테스트 TOKEN_TTL=3
확인 도구	개발자 도구 Network, curl -i, tools/check_api.py

T01~T17 결과 (17 건 전부 PASS)

ID	절차	실제 결과	판정	증거
T01	토큰 없이 목록·상세 조회	200, Authorization 헤더 없음	PASS	01
T02	Alice 로그인	200, expiresIn: 90, user(id 1), "Alice 님 로그인 중"	PASS	07
T03	틀린 비밀번호 / 없는 이메일	둘 다 401 INVALID_CREDENTIALS, 토큰 미발급	PASS	10
T04	토큰 없이 /me·글 등록	둘 다 401 TOKEN_REQUIRED	PASS	10
T05	Alice 토큰으로 작성 후 /me	201 및 Location, 작성자 Alice. /me 200 에 id·name·email 만	PASS	06, 03
T06	Alice 본인 글 수정·삭제	200(authorId·createdAt 유지, updatedAt 만 갱신), 204(본문 없음)	PASS	09, 13
T07	Bob 토큰으로 Alice 글 수정	403 FORBIDDEN, 재조회 시 모든 필드 동일	PASS	08, 05
T08	Bob 토큰으로 Alice 글 삭제	403 FORBIDDEN, 글 유지	PASS	11
T09	변조 토큰으로 /me·POST·DELETE	세 건 모두 401 INVALID_TOKEN, 데이터 변경 없음	PASS	05, 11
T10	만료 토큰으로 변경 요청	401 TOKEN_EXPIRED, 로그아웃 전환과 안내, 입력값 유지	PASS	02
T11	로그아웃 후 /me	401 TOKEN_REQUIRED, 헤더 미첨부	PASS	12
T12	등록 본문에 authorId / userId 추가	둘 다 400 BAD_INPUT, 글 생성 안 됨	PASS	11
T13	공백 제목 / 101 자 제목	둘 다 400 BAD_INPUT, 입력값 유지	PASS	04, 11
T14	없는 ID 조회·수정 / 잘못된 형식 ID	404 NOT_FOUND / 400 BAD_INPUT	PASS	10, 11

ID	절차	실제 결과	판정	증거
T15	서버 종료·재실행	재시작 전후 GET /api/posts 완전히 동일	**PASS**	14,15
T16	로그인 후 새로고침	로그아웃 상태 전환, 공개 조회는 200	**PASS**	01
T17	무토큰 수정 / Bob 본인 작업	401 TOKEN_REQUIRED / 작성 201(authorId 2), 수정 200	**PASS**	05, 13

응답 헤더·화면 추가 확인

- **WWW-Authenticate: Bearer**: 401 응답(TOKEN_REQUIRED, INVALID_TOKEN)의 헤더에 포함된 것을 curl -i 로 확인 (증거 05)
- **201 Location**: 등록 응답 헤더에 /api/posts/{id} 확인 (증거 06)
- **204 본문**: 응답 본문 없음, 화면에서 파싱 오류 없이 목록 갱신, 콘솔 오류 0 건 (증거 09)
- **401 과 403 의 화면 차이**: 401 은 로그아웃 상태로 전환, 403 은 "Bob 님 로그인 중" 유지. 403 직후 같은 토큰으로 /me 를 호출하면 200 이 반환되어 토큰 자체는 유효함을 확인 (증거 02, 08)
- **실패 후 데이터 불변**: 403(T07·T08)과 401(T09) 직후 재조회 시 제목·내용·authorId·createdAt·updatedAt 모두 동일 (증거 05)

자동 검사와 빌드

항목	결과
check_api.py --mode final	40 건 전부 통과 (exit 0), 출력에 토큰 원문 없음
TOKEN_TTL=3 + --expiry	48 건 전부 통과. 4 초 대기 후 만료 토큰 4 건 모두 401 TOKEN_EXPIRED
빌드·4173 실행	성공, API 요청이 8000 까지 전달됨
재시작 후 유지	GET /api/posts 응답 동일 (T15)

검사 도구는 일부 항목만 자동 확인하므로 위 수동 시나리오를 대체하지 않는다.

테스트 중 발생한 문제와 해결 과정

증상. 본인 글 수정(PUT)과 작성(POST)이 500 INTERNAL_ERROR 를 반환했다. 인증·권한·입력 검증 요청은 모두 정상이었고, 실패한 두 요청은 공통적으로 DB 에 쓰는 작업이었다. 수정은 응답이 500 인데 재조회하면 반영되어 있었다.

원인 추적. 같은 토큰으로 보낸 PUT /api/posts/999999 가 정상적으로 404 를 반환해 인증은 통과하고 있음을 확인하고 저장 단계를 의심했다. 오류 미들웨어가 500 을 JSON 으로 바꾸기만 하고 출력하지 않아 처음에는 원인을 좁히기 어려웠다.

원인. 코드가 아니라 실행 환경이었다. store.js 는 시작 시 DB 파일의 절대경로를 한 번만 정하는데, 서버 실행 중에 프로젝트 폴더를 옮겼다. store.update·store.create 는 메모리 DB 에 먼저 반영한 뒤 파일에 쓰는데, 이 파일 쓰기에서 ENOENT 가 발생해 500 이 났다. 404 요청은 저장까지 가지 않아 정상이었고, "반영된 것처럼" 보인 것도 메모리에만 반영된 것이었다. 별도 폴더에서 같은 상황을 재현해 동일한 ENOENT 와 메모리·파일 불일치를 확인했다.

조치와 재검증. 현재 경로에서 서버를 다시 실행하고, app.js 오류 미들웨어에서 500 일 때 console.error 로 오류를 출력하도록 보완했다(요청 헤더·본문은 출력하지 않아 토큰·키가 남지 않는다). 재실행 후 같은 두 요청이 각각 200 과 201 을 반환했고, check_api.py --mode final 40 건 통과를 유지했다.

남은 한계. 제공 코드의 store.js 는 파일 저장이 실패해도 메모리 변경을 되돌리지 않는다. 트랜잭션 처리가 필요해 과제 범위를 넘는다고 판단했다. clone 해서 실행하는 환경에서는 실행 중 폴더를 옮길 일이 없어 재현되지 않는다.

테스트 증거

캡처의 토큰 원문과 비밀 키는 가렸다. 파일은 docs/evidence/에 있다.

번호	파일명	내용
01	01_공개목록조회_200.png	로그아웃 상태 목록 조회 200, 헤더 없음
02	02_만료토큰_401.png	만료 토큰 401, 로그아웃 전환과 안내
03	03_me_Authorization 헤더.png	/me 요청의 Authorization: Bearer …
04	04_공백제목 400.png	공백 제목 400, 입력값 유지
05	05_curl_401_데이터불변.png	401 두 건, WWW-Authenticate, 재조회 결과
06	06_글등록 201.png	등록 201, 본인 글 버튼 표시
07	07_로그인 200.png	로그인 200, 사용자 표시
08	08_curl_403.png	Bob 이 Alice 글 수정 시 403
09	09_삭제 204.png	삭제 204, 목록 갱신
10	10 인증실패_404_400.png	T03·T04·T14 결과
11	11_curl_권한_입력검증.png	T08·T09·T12·T13 결과, 데이터 불변
12	12_로그아웃후_401.png	로그아웃 후 401 TOKEN_REQUIRED
13	13_curl_본인작업_200_201.png	재시작 후 수정 200, Bob 작성 201
14	14_T15_재시작전_종료.png	재시작 전 목록 5 건과 백엔드 종료 확인 (health HTTP 000)
15	15_T15_재시작후_diff 동일.png	재시작 후 health 정상, 목록 동일, diff 로 전후 응답 완전 일치

AI 활용과 참고 자료

AI 사용 여부: 사용

요구사항 정리와 작업 계획은 Claude 채팅으로, 코드 구현·디버깅·테스트는 Claude Code 로 진행했다. 모든 변경은 docs/worklog.md 에 파일·함수·출처·확인 결과를 단계별로 기록했고, 각 단계마다 직접 결과를 확인한 뒤 커밋했다.

도구 / 모델	목적	반영 위치	반영 방식
Claude 채팅 / Claude Opus 5	요구사항 정리, 작업 순서, 프롬프트 작성	작업 계획	참고
Claude Code / Claude Opus 5.5	백엔드 인증 구현	auth.js, .env.example	그대로 반영 (결과는 직접 확인)
Claude Code / Claude Opus 5.5	프론트 로그인·토큰·오류 처리	api.js, main.jsx, index.html	수정 후 반영 (문구는 본인 지정)
Claude Code / Claude Opus 5	500 오류 원인 분석	app.js 오류 미들웨어, 보고서 4 장	수정 후 반영 (조치는 본인 선택)
Claude Code / Claude Opus 5	curl·브라우저 시나리오 실행, 작업 기록	worklog.md, 보고서 3·4 장	참고 (결과는 직접 대조)

대표 사례 1 — 500 오류 원인 분석

항목	내용
도구 / 날짜	Claude Code / 2026-10-28
요청	본인 글 수정·작성이 500 인데 재조회하면 반영되어 있는 원인을 찾아 달라고 요청
AI 제안	서버 실행 중 폴더 이동으로 store.js 가 옛 경로에 저장하다 ENOENT 발생. 조치로 (1) 새 경로 재실행, (2) 500 로그 출력 제안
반영 위치	app.js 오류 미들웨어 / 보고서 4 장
직접 판단·수정	제안 중 (2)를 선택. store.js 구조 변경은 범위가 커서 한계로 기록하기로 결정
검증 결과	재실행 후 200·201 반환, DB 파일 갱신 확인. 시험 서버 로그에 ENOENT 스택이 남고 토큰·키는 없음을 확인. check_api.py 40 건 통과

대표 사례 2 — 로그인과 JWT 발급 (BE-01)

항목	내용
도구 / 날짜	Claude Code / 2026-10-28

항목	내용
요청	시작 코드의 issueToken 이 501 을 반환하는 상태에서, JWT_SECRET 을 64 자리 16 진수 난수로 설정하고 HS256·sub(문자열)·exp·iss·aud·TTL 90 초 조건으로 로그인과 토큰 발급을 구현하도록 요청했다. 서명은 라이브러리에 맡기고, 성공 200·틀린 계정 401 INVALID_CREDENTIALS·입력 오류 400 BAD_INPUT 으로 응답하되 키 값을 로그나 문서에 남기지 말 것을 조건으로 지정했다.
AI 제안	loadConfig()로 시작 시 키·TTL 검사, issueToken 에서 빈 페이로드 + 옵션 클레임으로 HS256 발급
반영 위치	auth.js loadConfig(), issueToken() / .env.example
직접 판단·수정	"abc" 같은 이메일에 app.js 형식 검사를 추가하지 않고 명세대로 401 로 두기로 결정
검증 결과	정상 로그인 200, 실패 401, 잘못된 입력 400, payload 에 iat·exp·aud·iss·sub 만 있는지 확인, 잘못된 키로 서버 시작 거부 확인

대표 사례 3 — 화면 문구와 응답 처리 (FE-03·04)

항목	내용
도구 / 날짜	Claude Code / 2026-10-28
요청	401 을 받으면 메모리의 토큰과 로그인 상태를 해제하고 code 별로 안내를 구분하되 403 은 로그아웃시키지 말 것, 204 는 JSON 을 파싱하지 말 것, 400·404 에서 입력값을 지우지 말 것을 조건으로 응답 처리를 요청했다. 함께 canEdit 을 작성자 비교로 바꾸고, 데모 배너와 미리 입력된 테스트 계정·비밀번호 안내를 제출본에서 제거하도록 요청했다.
AI 제안	code 별 안내 문구, 401 시 clearSession(), 404 처리, 작성자별 버튼, 화면 문구 초안
반영 위치	api.js request(), canEdit() / main.jsx App
직접 판단·수정	서비스 이름과 상단 소개, 카드 제목 등 화면 문구를 직접 확정
검증 결과	빌드본에서 문구 확인, 만료·변조·403·404·400 화면 확인, dist 에 테스트 비밀번호 없음 확인

참고 자료

자료	사용 위치	반영 내용
과제 안내서 (docs/01_과제공지.md)	전체	요구사항, 응답 코드 기준
API 명세 (docs/02_API 명세.md)	auth.js, app.js	iss·aud 기본값, 입력 길이 규정
테스트 시나리오 (docs/05_테스트시나리오.md)	4 장	T01~T17 절차
제공 시작 코드	전체	유지 부분은 변경 내역표에 표시

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

본문의 authorId 를 믿으면 클라이언트가 남의 ID 를 넣어 다른 사람 이름으로 글을 쓰거나, 수정 시 작성자를 바꿔 글을 가로챌 수 있다. 내 코드는 등록 시 작성자를 app.js 의 POST 라우트가 currentUserId(req)로 얻은 값, 즉 토큰을 검증하고 실제 계정까지 확인한 ID 로 정한다. 본문은 fields(req.body, {title, content})를 거치므로 authorId·userId·id 같은 추가 키가 있으면 400 BAD_INPUT 으로 거부되고 글이 만들어지지 않는다(T12). 수정 SQL 도 title·content·updated_at 만 바꾸므로 작성자는 바뀌지 않는다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼 숨김은 화면에서만 일어나는 처리라서 curl 이나 API 클라이언트로 PUT·DELETE 를 직접 보내면 그대로 우회된다. 실제 검사는 auth.js 의 assertOwner(userId, authorId)가 한다. PUT·DELETE 라우트에서 currentUserId 로 검증한 ID 와 DB 에 저장된 글의 authorId 를 비교해 다르면 403 FORBIDDEN 을 던지고, DB 변경 코드는 실행되지 않는다. Bob 토큰으로 curl 을 이용해 Alice 글을 수정·삭제해 403 과 데이터 불변을 확인했다(T07, T08).

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 payload 를 base64 로 풀어 읽기만 하므로 서명이 틀리거나 내용이 바뀌어도 그대로 읽힌다. verify 는 키로 서명을 확인하고 알고리즘·만료·iss·aud 까지 검사한다. 내 currentUserId 는 decode 를 쓰지 않고 jwt.verify 에 algorithms: ["HS256"], issuer, audience 를 지정한다. 여기에 더해 라이브러리가 건너뛰는 exp 존재 여부와 sub 형식을 직접 확인하고, findPublic 으로 계정이 실제로 있는지 조회한 뒤 조회된 user.id 를 반환한다. 만료는 TOKEN_EXPIRED, 나머지는 INVALID_TOKEN 으로 구분한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

상황	바뀌는 것	서버 쪽
로그아웃	이 탭의 메모리에서 토큰·사용자를 지우고 화면과 폼을 비움	변화 없음
토큰 만료	exp 가 지나 서버가 401 TOKEN_EXPIRED 를 반환, 프론트가 로그아웃 상태로 전환	verify 가 거부
새로고침	모듈 변수가 초기화되어 로그아웃 상태. 글 데이터는 SQLite 에 있어 그대로	변화 없음

이 서버는 토큰 목록을 저장하지 않는 무상태 방식이라 로그아웃은 브라우저에서 토큰을 버리는 것일 뿐이다. 그래서 이미 복사된 토큰은 exp(기본 90 초)가 지나기 전까지 계속 유효하다. TTL 을 짧게 둔 것이 이 위험을 줄이는 방법이다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

본인 글 수정·작성이 500 을 반환하는데 재조회하면 반영되어 있던 오류가 가장 어려웠다. 같은 토큰으로 없는 글을 수정하면 404 가 정상적으로 나와서 인증이 아니라 저장 단계 문제로 좁혔다. 원인은 서버 실행 중 프로젝트 폴더를 옮긴 것이었다. store.js 가 시작 시 정한 옛 절대경로에 쓰다 ENOENT 가 났고, 메모리 DB 에는 먼저 반영되어 성공한 것처럼 보였다. 새 경로에서 서버를 다시 실행하고, 같은 일이 생기면 원인이 바로 보이도록 500 일 때 서버 로그에 오류를 출력하게 고쳤다. 재검증으로 같은 요청이 200·201 을 반환하고 DB 파일이 갱신되는 것, 시험 서버 로그에 ENOENT 스택이 남는 것, check_api.py 40 건 통과를 확인했다.

남은 문제와 개선 방향

남은 문제	내용	다음 개선
저장 실패 시 메모리 불일치	store.js 는 파일 저장이 실패해도 메모리 변경을 되돌리지 않는다 (제공 코드 구조)	트랜잭션 또는 롤백
의존성 취약점	npm audit 에서 body-parser(high), qs(moderate) 보고. 제공 코드 동작이 바뀔 수 있어 미조치	호환성 확인 후 갱신
보완 전 동작 미확인	JSON 아닌 실패 응답 처리를 보완했으나 보완 전 화면 문구는 미확인	—
로그아웃 후 토큰 유효	무상태 JWT 라 복사된 토큰은 만료 전까지 유효	리프레시 토큰·차단 목록 검토

제출 확인

☐ 실제 코드와 설명 일치 (줄 번호는 커밋 691db35 기준)

☐ 테스트 직접 수행 ☐ AI 활용 및 출처 기록 (모델명 기입 필요)

☐ 비밀 키와 토큰 제거 (캡처 가림 확인) ☐ 루트 README 작성

☐ 보고서 PDF 와 증거 업로드 ☐ README 문서 링크 확인

☐ 교수자 접근 가능 ☐ 저장소 URL 과 커밋 SHA 제출